

hello.c 源程序 文本文件 以字节为序 只由ASCII字符构成
编译 链接 可执行目标程序
hello.c → CPP 编译 → hello.i
hello.i → CCL 编译 → hello.o
hello.o → AS 汇编 → hello.o
hello.o → LD 链接 → hello

二. 信息的表示与处理
除基取余 2/835 ... 1 0.48750 = 1.375 ... 1
乘基取整 2/1.1 ... 1 0.375 * 2 = 0.75 ... 0
0.5 * 2 = 1 ... 1
gcc-m32 prog.c 32位编译
连续字节 → 找字中最低地址 大端 最高 ...

位运算: ~ & | ^ 逻辑运算, || && ! (前两个结果不算后)
移位: x << k 左移k位, 补0; x >> k 右移k位, 补0
x >> k 逻辑/算术

$B2U(x) = \sum_{i=0}^{w-1} x_i 2^i$ $B2T(x) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$
 $UMax_w = 2^w - 1$ $TMax_w = 2^{w-1} - 1$
 $TMin_w = -2^{w-1}$
 $T2U(x) = \begin{cases} x + 2^w, & x < 0 \\ x, & x \geq 0 \end{cases}$ $U2T(x) = \begin{cases} u, & u \leq TMax_w \\ u - 2^w, & u > TMax_w \end{cases}$

强制转换: ① C, ② 赋值 ③ %d %u %s ④ 运算 → 无符号
char 1 1 截断: 丢高 w-k 位, 溢出
short 2 2 U: $x' = x \bmod 2^k$
int 4 4 T: $x' = U2T(x \bmod 2^k)$
long 4 8
float 4 4
double 8 8
* 4 8
2.3. 整数运算
① U加法: $x + y = \begin{cases} x+y & < 2^w \\ x+y-2^w & \geq 2^w \end{cases}$
判定溢出: $S = x + y < x$
求反: $-x = \begin{cases} x & x=0 \\ 2^w - x & x>0 \end{cases}$

② T加法: $x + y = \begin{cases} x+y & < 2^{w-1} \\ x+y-2^{w-1} & \geq 2^{w-1} \end{cases}$
判定: $x > 0, y > 0, S < 0$
 $x < 0, y < 0, S > 0$
③ 补码非: $-x = \begin{cases} TMin_w - x & x = TMin_w \\ -x & \text{其他} \end{cases}$
④ U乘法: $x * y = (x * y) \bmod 2^w$
⑤ 补码乘: $x * y = U2T((x * y) \bmod 2^w)$

⑥ 除以2幂: 逻辑/算术移 负数: 补码左移 (移位)
⑦ 除以2幂: 逻辑/算术移 负数: 补码左移 (移位)
2.4. 浮点数: $V = (-1)^S * M * 2^E$ 符号, 尾数, 阶码
float: 1+8+23 double: 1+11+52 Exp
⑧ 规格化 (E不全0, 不全1) -126 ~ 127
阶 E = Exp - Bias ($2^{k-1} - 1$) -1022 ~ 1023

硬件: 1. 总线, 2. I/O, 3. 主存, 4. 处理器
⑨ 非规格化 (阶码全0) 0.1 ~ 2^{-1}
E = 1 - Bias M = f 设计可整型排列
⑩ 特殊: 阶码全1, 小数0 → 无穷大
⑪ 舍入: 向偶数/最低有效位舍入

2.5 转换
int → float 舍入 int/f → d 精确
d → f 舍入 f/d → int 向0舍入/溢出
三. 机器级表示
IA32 → x86-64
3.2. 程序编译 gcc -Og -o p.pl.c p2.c
① 机器级 ISA 指令集架构
1° PC 程序计数器 %rip 2° 整数寄存器 16 64 位
3° 条件码寄存器 4° 向量寄存器 (浮点)
5° 程序内存: 代码, 操作系统信息, 栈, 内存块地址
6° 地址 64 位字, 高 16 → 0
② -s 汇编代码 -c 生成 .o 1.15 byte, 省略
③ 反汇编 objdump -d ... 0 结尾 'q', 实际可
④ nop 是为使变为 16 字节 (无意义) 执行 main, 变大, 代码
包括启动, 终止, 交互

3.3. 数据格式
b (字节) w L q s (单精) l (双精)
1 2 4 8 4 8 x8
⑤ 访问信息
① CPU 通用寄存器 16 * 64 4 字节 → 高 16 位不变
%rax %eax %ax %al %rbp %ebp %bp %bpl
%rdx %edx %dx %dil %rsp %esp %sp %spl
%rsi %esi %si %sil %r8 %r8d %r8w %r8b
%r15 %r15d %r15w %r15b

② 操作数: 立即数 (常数) \$ 整数
寄存器 内存引用 (有效地址) 比例地址 ~
寻址模式: 立即数 ~ 寄存器 ~ 绝对 ~ 间接 ~
③ 1° 传送 movq { Imm, Reg, Mem }
2° 条件码 movxx { Reg, Mem }
3° 符号扩展 movsb, movsw, movslq

④ 算术: 立即数 (常数) \$ 整数
寄存器 内存引用 (有效地址) 比例地址 ~
寻址模式: 立即数 ~ 寄存器 ~ 绝对 ~ 间接 ~
⑤ 1° 传送 movq { Imm, Reg, Mem }
2° 条件码 movxx { Reg, Mem }
3° 符号扩展 movsb, movsw, movslq

⑥ 算术: 立即数 (常数) \$ 整数
寄存器 内存引用 (有效地址) 比例地址 ~
寻址模式: 立即数 ~ 寄存器 ~ 绝对 ~ 间接 ~
⑦ 1° 传送 movq { Imm, Reg, Mem }
2° 条件码 movxx { Reg, Mem }
3° 符号扩展 movsb, movsw, movslq

② movzbl 0 扩展
movsbl 符号扩展
cltq (%eax) → %rax 带符号扩展
*(%ebx) 不等于, 地址 64 位
④ 压入, 弹出栈 %rsp
pushq, popq 按原地址
栈顶位置低, 向下生长
入: rsp-8 再传送
出: 传送再 rsp+8

⑤ 算术, 逻辑操作
leaq (%eax), %rax
inc D, D+1 } 一元
dec D, D-1 } 操作
neg D, -D
not D, ~D
add S, D %eax + D → D
sub D, S → D
imul D * S → D
xor D ^ S → D
or D | S → D
and D & S → D
sal k, D D << k
shl D << k
sar D >> k 算术
shr D >> k 逻辑

1° leaq 可执行有限 +, x
2° 一元: 既可寄存, 内
二元: 第二元既寄存又用
3° 移位
k 可 Imm, 也可 %clt
对 w 位长操作 $2^m = w$
4° 特殊

3.6 控制
① 条件码
CF 进位 无符号溢出
ZF 0 0
SF 符号 负
OF 溢出 有溢
② 分支/传送
jmp 直接 (地址) 跳转
后面同 set 指令: PC 相对
③ 循环
do-while
while
④ switch 在 .rodata 中
⑤ 过程
调用控制
数据传递
管理局部数据

callq f- %rip - PC
retq (传参) 2
顺序: %rdi %rsi %rdx %rcx
%r8 %r9
返回: %rax 调用者保存
大于 6 个栈, 7 位于寄存器
%r10 ~ %r11

3.7 过程
调用控制
数据传递
管理局部数据
callq f- %rip - PC
retq (传参) 2
顺序: %rdi %rsi %rdx %rcx
%r8 %r9
返回: %rax 调用者保存
大于 6 个栈, 7 位于寄存器
%r10 ~ %r11

leaq 不改变条件码, 其余会
1° 逻辑指令: CF, OF=0
2° 移位: CF 是最后一个移出 OF=0
3° inc, dec 设 DF, ZF 不破 CF
指令 cmp 只设条件码 S₂-S₁
S₁, S₂ bwlq 相当于 sub
test 相当于 and (rax, rax)
② 访问条件码 (一个字节, 然后
1° 根据条件码改位 (数
2° 条件跳转
3° 条件传送
jmp set e D ZF =
ne ~ ZF ≠
s SF 负
ns ~ SF 非负
g ~ (SF ^ OF) ^ ZF 有
ge ~ (SF ^ OF)
l (SF ^ OF) | ZF
a ~ CF & ~ ZF 无
ae ~ CF
b CF
be CF | ZF
设置一个字节
③ 跳转 *(%rax) * %rax
jmp 直接 (地址) 跳转
后面同 set 指令: PC 相对
④ 分支/传送
⑤ 循环
do-while
while
⑥ switch 在 .rodata 中
⑦ 过程
调用控制
数据传递
管理局部数据

被调用者保存 %rbx, %rbp, %rsp, %r12 ~ %r14
(栈帧指针) → 离开前恢复
3.8. 数组
① 原理: L-N 字节连续区域
指针: E int * Xe
E[0] int M[Xe]
E[i] int M[Xe+i]
%E[i] * Xe + i
E+i-1 * Xe + i-1-4
*(E+i-3) int M[Xe+i-2]
%E[i]-E long i
② 嵌套
③ 定长, 变长数组 struct
3.9. 变长数据结构 Union
一段连续区域, 偏移, 对齐
前: k 字节地址必须 k 倍
malloc 16 倍 栈帧 16 倍
后: 大小, 当最大块
容量: 工作集 > 缓存
类型: 数组, 结构, 枚举, 堆, 栈
寄存器 4-8 CPU 寄存器 0 寄存器
TLB 表 4-8 CPU 寄存器 0 MMU
L1 64 字节 L2 4 字节
L3 64 字节 L4 10 字节
命令: d 在大层中存储
不命中: k+1 取来 → k, 已满了
替换: 最近 → 牺牲块
6.4. Cache 高速缓存
① 寄存器: 地址 16 位, M=2¹⁶
S=2⁵ 组 E 路行块 B=2⁶
C=S * E * B m-bits s b
标记 组索引 块索引
直接映射: (每组 1 个)
1° 冲突不命中: 抖动现象
组相联 KE < C/3
全相联 E = C/3
昂贵, x 非空位
写: 直写: 将 1 个块写进缓存
写回: 推迟写入内存
至 1 个要替换 (需要 1 个修
改)

ASLR 地址空间布局随机化
2. 栈破坏检测
金丝雀: %fs, 40 (字节)
3. 限制可执行代码区域
② 变长栈帧 %rbp 帧指针
leave 恢复 rbp
五. 优化程序性能
5.1 优化编译器
-Og -O1 -O2 -O3 优化
内存到寄存器使用 函数调用
内存到寄存器使用 函数调用
内存到寄存器使用 函数调用
5.2 优化 CPU 优化
① 代码块移动 Cache 友好
② 复杂运算简化 (C, C++)
③ 共享公用子表达式
5.3 异常处理
① 函数调用 写回已
写回已

ASLR 地址空间布局随机化
2. 栈破坏检测
金丝雀: %fs, 40 (字节)
3. 限制可执行代码区域
② 变长栈帧 %rbp 帧指针
leave 恢复 rbp
五. 优化程序性能
5.1 优化编译器
-Og -O1 -O2 -O3 优化
内存到寄存器使用 函数调用
内存到寄存器使用 函数调用
内存到寄存器使用 函数调用
5.2 优化 CPU 优化
① 代码块移动 Cache 友好
② 复杂运算简化 (C, C++)
③ 共享公用子表达式
5.3 异常处理
① 函数调用 写回已
写回已

ASLR 地址空间布局随机化
2. 栈破坏检测
金丝雀: %fs, 40 (字节)
3. 限制可执行代码区域
② 变长栈帧 %rbp 帧指针
leave 恢复 rbp
五. 优化程序性能
5.1 优化编译器
-Og -O1 -O2 -O3 优化
内存到寄存器使用 函数调用
内存到寄存器使用 函数调用
内存到寄存器使用 函数调用
5.2 优化 CPU 优化
① 代码块移动 Cache 友好
② 复杂运算简化 (C, C++)
③ 共享公用子表达式
5.3 异常处理
① 函数调用 写回已
写回已

ASLR 地址空间布局随机化
2. 栈破坏检测
金丝雀: %fs, 40 (字节)
3. 限制可执行代码区域
② 变长栈帧 %rbp 帧指针
leave 恢复 rbp
五. 优化程序性能
5.1 优化编译器
-Og -O1 -O2 -O3 优化
内存到寄存器使用 函数调用
内存到寄存器使用 函数调用
内存到寄存器使用 函数调用
5.2 优化 CPU 优化
① 代码块移动 Cache 友好
② 复杂运算简化 (C, C++)
③ 共享公用子表达式
5.3 异常处理
① 函数调用 写回已
写回已

4. TLB, 流水线, 多级倒栽梭为
 执行 (不可见) 有效 →
 CPU 提供硬件支持, OS 控制
 主存 = 物理内存
 cache 为各级的缓存, TLB 为缓存
 编译器将数据转换为相应的二
 进制码/补码, 类型不一致时补
 码用快速缓冲存储器作为页
 表的 Cache. TLB 中保存最近
 常用的虚拟页号-物理页号

编译原理：由下而上补全量
 连接PTB符号和已配，并和4级
 表后，一级表大量条目其内容
 为NULL，同样二，三级表也如此
 样，只有已配二级表符号和4级表
 的。执行程序44，最了3次缺空出
 代器，数据，谁栈
 常量 → 只读代石段
 全相联Cache不发生冲突不十
 下，下 指到旧 → 找表为

每一条指令 $\rightarrow VP0 = PP0 = 12$
 $VP0 - UP0$
 $PPU = 48 - 12 = 36$

取4KB，一个来算8个字节
 $VPNU = \frac{36}{4} = 9$
 for (int i=0; i<VPNU; i++)

及原有 $\frac{2^{12}}{2^2} = 2^9$ 项

FLB I 因素引数 (128条指令, 4路) $\rightarrow 128 \times 4 = 512$
 FLBT 系数1的
 $CT + CI + CO = 52$
 $\frac{CI}{\text{级}}$ $\frac{CO}{\text{级}}$

每个指令为取指译码...会超过限制
 System Call
 异常
 无硬件中断

CPU 附近 RIP 取指 (F-指令)
 信号不安全, printf, sprintf, malloc, exit

17. 对管道进行异步发送 SIGCHLD 信号

- ① 检查第一个参数是否应当为置 shell 命令
- ② 若非内部，调用 fork()
- ③ 调用 setpgid 函数，调用 execve() 执行
- ④ 若要后台 (&)，用 wait 等待
- ⑤ 若后台，则 shell 返回